

adds to your portfolio as a software engineer, unless you participate in triaging and fixing some issues in the first case. However, participating in the review process of code and translations can be super helpful, especially considering the incidents where attackers managed to get their malicious code and translations accepted into free software projects (the xz incident of 2024 and hate speech found in Ubuntu 23.10).

Also, avoid making too many cheap submissions. Yes, it is important to get all the typos fixed in the README file, and this is a good starting point if you are testing the waters but avoid filing a pull request (or sending a patch) every day that fixes a single typo. Combine such contributions so that you can fix issues without annoying others.

Where do I join?

Even a person who hasn't thought about making his or her first contribution knows one could find and contribute to free/open source projects on *github.com*. Some people might be familiar with *gitlab.com* as well. However, not all open projects are hosted on GitHub or the like. Some projects self-host GitLab, i.e., they install the GitLab software on their server, making an environment similar to *gitlab.com* available under their domain (e.g., *gitlab.xfce.org*). Some projects use much lighter interfaces while some don't have a web Git interface at all. Some projects use Git for version control, but they may not employ the fork-PR-merge model (explained later). Instead, you'll have to create a patch from your commits and send it as an email to the maintainers.

Check the website to find out where development happens for a particular project, if there is one. To make sure you are not visiting a phony website or such a repo, you can depend on the package descriptions found on your trusted package store.

On GitHub or GitLab, there is no need to join a project as long as

you have an account on the platform and you are planning to submit your contributions as an external contributor.

Many projects host discussion forums for both users and developers to participate in, where both technical and non-technical subjects are discussed. It's a good idea to join them — *discourse.gnome.org* is an example.

Several projects still run old school mailing lists, which work very well. Discussions are threaded and archived. Patches are also sent to these lists. Although the archives are public, participation may require free but explicit subscription.

Many projects have a dedicated bug tracking facility, like *bugzilla.mozilla.org*. You may need an account there as well.

Before you jump into contributing, make sure to check the guidelines of each project. They not only list the technical instructions, but code of conduct and information regarding copyright transfer as well. Most projects let you own the copyright of the code you've contributed, as long as you let them distribute the code under the licence the project is in. However, some projects could ask you (and the organisation you work for) to sign a copyright transfer agreement to avoid disputes in future.

The contribution workflow

Contributing over GitHub usually starts with an issue, which is a bug report or a feature request. You can pick an existing issue or file a new one. This is done upstream itself. Upstream means the original repository of the project you are trying to contribute to. Once the issue is filed, make sure nobody else is working on it and then start. But if you are an outsider to the project (i.e., you don't have commit access), you'll have to first fork the upstream. This fork is where you'll make the changes. Once you're done, you can file a pull request to the upstream, and they'll merge your changes if they are satisfied with it.

Although this sounds too technical, it's as simple as participating in a discussion forum, thanks to the user-friendly web interfaces these platforms provide.

(You don't have to fork each time to make a contribution. Once you've forked a repo, you can use the same fork to make as many contributions as you want. Just make sure to sync it with the upstream before you start working.)

You can easily create the projects that use Git as the version control system but accept contributions from strangers as patches using the command *git diff*. A patch is a piece of text that contains the changes you've made, which can be mailed to the upstream developers as an attachment or as the body of the mail.

Keep in mind that the exact workflow can vary from project to project. For example, some projects will require you to format pull requests in a certain way, with the related issue number quoted in the title. Others may happily accept your pull request without a related issue being filed at all. Be sure to check a project's contribution guidelines before proceeding.

How UI translation is done

Translating user interfaces and web pages from English to Malayalam during my high school days is how I got into contributing to free software. I don't think translating a web page requires any explanation. Let's skip to the UI (user interface) translation of applications.

Unlike for documents, UI translation is not done on a screen-by-screen basis. Rather, you translate individual strings (aka messages). This ensures consistency and avoids rework for strings like 'OK' that are being used in several places.

Nowadays the UI translation of many projects is done via some interactive online platform like *weblate.org* or *transifex.com*. All you have to do there is join a project and then type-in (or review) the translation of strings one by one.